Connecting to a Shared Lab Cluster

This guide shows you how to connect to a lab cluster that a teammate launched and shared with you, and what you can do once you are connected. You do **not** need an AWS account, Terraform, or Docker. You only need `kubectl` and one file from the person who launched the cluster.

What you will receive

The person who launched the cluster gives you two things:

- **A kubeconfig file**, for example `team.kubeconfig`. It holds the cluster address and a login token.
- **The lab domain**, for example `example.com`. The five applications are at `https://<app>.<lab-domain>`.

The file is a credential. Anyone who has it can read the cluster, so keep it within your team and never post it publicly.

What you can and cannot do

Your access is **read-only**. You can look at everything, and you cannot change anything.

You CAN	Example
List pods, Services, Deployments, Ingresses, and autoscalers in every namespace	`kubectl get pods -A`
See nodes and their resource usage	`kubectl get nodes`, `kubectl top nodes`
Read logs and events	`kubectl logs -n banking deploy/banking-backend`
Inspect any object in detail	`kubectl describe pod -n ecommerce <pod>`
Read ConfigMaps, quotas, and Endpoints	`kubectl get configmaps -n banking`
Open the five applications in a browser	`https://banking.<lab-domain>`

You CANNOT	What you will see
Read any Secret (database passwords)	`Forbidden`
Create, edit, scale, restart, or delete anything	`Forbidden`
Open a shell in a pod (`exec`) or port-forward	`Forbidden`
Change nodes, namespaces, Ingresses, or permissions	`Forbidden`

`Forbidden` is the access model working correctly. It is not an error on your side.

Step 1. Install kubectl

Windows (PowerShell):

```
winget install -e --id Kubernetes.kubectl
```

macOS:

```
brew install kubectl
```

Linux and other options: https://kubernetes.io/docs/tasks/tools/

Open a new terminal and check that it is installed. Any recent version works, and versions 1.32 to 1.34 match this cluster best.

```
kubectl version --client
```

Step 2. Save your kubeconfig file

Save the file you received as `lab.kubeconfig` inside a `.kube` folder in your home directory. Create the folder if it does not exist.

- Windows: `C:\Users\<you>\.kube\lab.kubeconfig`
- macOS and Linux: `~/.kube/lab.kubeconfig`

Do not save it inside a Git repository folder.

Step 3. Point kubectl at the file

Run one of these in every new terminal window.

```
# macOS, Linux, WSL, Git Bash
export KUBECONFIG=~/.kube/lab.kubeconfig
```

```
# Windows PowerShell
$env:KUBECONFIG = "$HOME\.kube\lab.kubeconfig"
```

Step 4. Check that it works

```
kubectl get nodes
kubectl get pods -A
```

You should see a list of nodes with the status `Ready`, and a list of pods with the status `Running`. If you see that, you are connected.

To confirm you are read-only, try to create something. This must fail:

```
kubectl run test --image=nginx
```

The result `Forbidden` means everything is set up correctly.

The five applications

Each application has its own namespace with a frontend and a backend Deployment.

Application	Namespace	Deployments	URL
E-commerce	`ecommerce`	`ecommerce-frontend`, `ecommerce-backend`	`https://ecommerce.<lab-domain>`
Banking	`banking`	`banking-frontend`, `banking-backend`	`https://banking.<lab-domain>`
Food delivery	`food-delivery`	`food-delivery-frontend`, `food-delivery-backend`, `food-delivery-redis`	`https://food-delivery.<lab-domain>`
Student portal	`student-portal`	`student-portal-frontend`, `student-portal-backend`	`https://student-portal.<lab-domain>`
Support tickets	`support-tickets`	`support-tickets-frontend`, `support-tickets-backend`	`https://support-tickets.<lab-domain>`

Other namespaces you can look at: `kube-system` (Kubernetes components and the load balancer controller), and `default`.

Commands to explore

To see	Command
Everything running, all namespaces	`kubectl get pods -A`
More detail (node, IP)	`kubectl get pods -A -o wide`
Services and their ports	`kubectl get services -A`
Deployments and replica counts	`kubectl get deployments -A`
How traffic enters the cluster	`kubectl get ingress -A`
Autoscaling	`kubectl get hpa -A`
One app, everything	`kubectl get all -n banking`
Which pods sit behind a Service	`kubectl get endpoints -n ecommerce`
Recent events, newest last	`kubectl get events -n banking --sort-by=.lastTimestamp`
Full detail on a pod	`kubectl describe pod -n ecommerce <pod-name>`
A pod's logs	`kubectl logs -n ecommerce <pod-name>`
Logs of a Deployment	`kubectl logs -n ecommerce deploy/ecommerce-backend`
Logs from before a crash	`kubectl logs -n ecommerce <pod-name> --previous`
CPU and memory use	`kubectl top pods -A` and `kubectl top nodes`
Resource limits per namespace	`kubectl get resourcequota -A`
An object as YAML	`kubectl get deployment -n banking banking-backend -o yaml`

If someone runs an incident

The person who launched the cluster may break one of the applications on purpose so your team can diagnose it. You cannot fix it, but you can find the cause. Work in this order:

- Find what is unhealthy: `kubectl get pods -A` (look for `CrashLoopBackOff`, `ImagePullBackOff`, `Pending`, `0/1` ready, or many restarts).
- Read the events for that namespace: `kubectl get events -n <namespace> --sort-by=.lastTimestamp`.
- Describe the failing pod and read the `Events` and `Last State` sections: `kubectl describe pod -n <namespace> <pod>`.
- Read its logs, and the previous logs if it restarted: `kubectl logs -n <namespace> <pod> --previous`.
- Check the Deployment, Service endpoints, and Ingress for something that does not match.
- Write down the root cause and the fix you would apply, then compare notes with the person who triggered it.

Troubleshooting

What you see	What it means and what to do
`Forbidden` on create, edit, delete, or Secrets	Expected. You are read-only.
`Unauthorized`	Wrong file, or your access was revoked. Ask for the current file.
`Unable to connect to the server`, timeout, or `no such host`	The cluster is stopped or rebuilt, or your network blocks it. Check with the person who launched it and try another network.
An error that mentions `localhost:8080`	`KUBECONFIG` is not set in this terminal, so kubectl found no cluster. Repeat Step 3.
`kubectl: command not found`	Install kubectl (Step 1) and open a new terminal.
The file name shows `.txt` at the end	Windows added an extension. Rename it to `lab.kubeconfig`.

Rules for using the shared file

- Treat the kubeconfig like a password. Do not post it in public chats, forums, screenshots, or Git repositories.
- Do not share it with people outside your team.
- The cluster is shared by your whole team. Close long-running commands such as `kubectl get pods -w` and `kubectl logs -f` when you are finished.
- Do not try to bypass the read-only limits.